Research Project Report

DocuStruct

The requirement for the award of the Degree of

MASTER OF SCIENCE

(INFORMATION TECHNOLOGY)

By

Mr. Shivam Vishwakarma

Under the esteemed guidance of

Ms. Aditi Prajapati

DEPARTMENT OF INFORMATION TECHNOLOGY

RAMNIRANJAN JHUNJHUNWALA COLLEGE (AUTONOMOUS)

(Affiliated to University of Mumbai)

Ghatkopar West, Mumbai-400086

Academic Year

2025-2026

Hindi Vidya Prachar Samiti's

Ramniranjan Jhunjhunwala College of Arts, Science & Commerce, Ghatkopar (W)

(Empowered Autonomous)

Affiliated to
UNIVERSITY OF MUMBAI

This is to certify that the research project entitled DocuStruct, is Bonafide work of Mr. Vishwakarma Shivam Suresh Sushila, bearing Seat No. 6709 submitted in partial fulfillment of the requirements for the award of the degree of Master of Science in Information Technology from R.J. College of Arts, Science and Commerce (Empowered Autonomous) affiliated to University of Mumbai during academic year 2025 - 2026.

Project Guide

PG Co-ordinator

External Examiner

8 APR 2026
EXAMINED

College seal

ACKNOWLEDGEMENT

The success and outcome of this project were possible by the guidance and support from many people. We are incredibly privileged to have got this all along with the achievement of my project. It required a lot of effort from each individual involved in this project with us and we would like to thank them.

We appreciate and thank our **PG-In charge Mrs. Bharati Bhole**, for granting us an opportunity to do the project activity and providing us with all support and leadership, which made us finish the project duly. We would like to give a very special honor and respect to our teacher and Project Guide, **Prof. Aditi Prajapati** who took a keen interest in checking the minute details of the project worked and guided us throughout the same.

Then we would like to thank our friends and individuals who have helped us with their valuable suggestions and guidance and have been helpful in various phases of the completion of the project.

- **Shivam Vishwakarma**

TABLE OF CONTENT

Sr no.		Particulars	Page no
1		Abstract	1
2		Problem Statement	2
3		Introduction	4
4		Project Objectives and Proposed System	5
5		Literature Survey	8
6		Literature Review	10
7		Scope of Project and Expected Outcomes	12
	7.2	Expected Outcomes	13
8		Tools and Technologies	14
9		Project Design	20
	9.1	System/Project Design	24
	9.2	Activity/UML Diagrams	27
10		Project Development and Implementation	34
	10.1	Project Development Steps with Software Code	35
	10.2	Project Testing	29
11		Result and Conclusion	44
	11.1	Software Result Screenshots	46
	11.2	Conclusion	53
12		References	54

1. ABSTRACT

In the modern business environment, organizations deal with a large volume of invoices and financial documents on a daily basis. Extracting the structured information from these documents is a critical step for accounting and auditing. However, invoice formats may vary widely across different vendors & customers, which makes the extraction of tabular information a challenging task.

This research project presents a possible solution, i.e., DocuStruct, a system designed to detect tables in invoice documents and reconstruct structured tabular data from scanned images or digital files. The proposed system combines the process of document layout detection, optical character recognition (OCR), structural analysis, and intelligent row–column reconstruction techniques to transform raw invoice images into structured datasets suitable for further analysis and storage.

The system utilizes a layout detection model trained on document layout datasets to identify the table regions within the invoices. Once the table region is detected, OCR is utilized to extract the textual content along with its spatial information. The extracted text data are then analyzed to determine row boundaries and column alignments using clustering and other geometric methods. The logical row reconstruction process merges the extracted text segments to form complete records (rows). The final structured table is exported into a spreadsheet (.xlsx), enabling easy integration with the existing business workflow environment.

The developed system also incorporates the confidence analysis metrics that provide quick insights into the reliability of extracted information. These metrics assist users in evaluating the quality of results and identifying potential errors for manual verification. The proposed solution demonstrates that combining layout detection techniques with spatial analysis of OCR outputs can significantly improve the accuracy and reliability of invoice table extraction systems.

2. PROBLEM STATEMENT

Businesses and organizations process invoices from multiple vendors who each use their own distinct formats. Invoices use tables to display item descriptions and quantities and prices and totals, but there is no universal standard method for displaying these tables. The process of automatic invoice data extraction remains difficult because of this issue.

The traditional methods depend on manual data entry or basic optical character recognition (OCR) systems. OCR technology enables text image conversion into machine-readable characters, but it lacks the ability to comprehend the structural connections between different text elements. The output from OCR systems produces unstructured word sequences that do not show how words connect to each other in table structures.

Companies face another challenge arising from the diversity of invoice layouts, which creates obstacles to their operations. Tables may appear with visible borders, partial borders, or no borders at all. Some invoices contain multiple tables, summary sections, or multi-line content that spans across several rows. Rule-based extraction techniques face challenges when dealing with table variations because they depend on fixed table formats.

Furthermore, inaccuracies in OCR recognition, variations in text alignment, and image noise can lead to fragmented or misplaced data elements. These issues complicate the reconstruction of table structures from OCR results. Due to these limitations, organizations often rely on manual verification processes after automated extraction, which reduces efficiency and increases operational costs.

Therefore, there is a need for a robust and adaptable system that can automatically detect tables within invoice documents, accurately reconstruct their row–column structure, and convert them into structured data formats suitable for downstream processing.

The goal of this research project is to design and implement such a system by integrating document layout analysis, OCR technology, and structural reconstruction logic. The system should be capable of handling diverse invoice layouts while maintaining high accuracy and reliability in the extracted data.

3. INTRODUCTION

Every business, regardless of its size, deals with invoices on a daily basis. The documents contain vital details, which include item descriptions and quantities and prices and taxes and totals, which need to be documented precisely for maintaining proper financial operations. For years, this work was done manually, with employees reading through each invoice and typing the relevant details into accounting systems. The process worked properly for smaller operations but became unmanageable when the volume of invoices increased, which resulted in delays and mistakes during data entry and extra expenses for the company.

The system used OCR technology to transform scanned invoice images into text that people could read. The ability to read text does not provide sufficient information. Invoices present their data through tables, which require understanding their table structure for extracting data from the tables.

The creation of invoices gets complicated because every invoice has its unique design. Different vendors use different layouts, fonts, and table styles. Some tables show their borders with clear grid lines, while other tables use spacing and alignment to create column divisions. The existing system fails to provide a dependable extraction method because it requires continuous human modification for processing all invoice types.

The project directly addresses the issue by creating a system that can read invoice text and comprehend the way that text is organized. The system identifies table areas in the document, evaluates how each text component is distributed throughout the document, and uses this information to create rows and columns that make sense.

4. PROJECT OBJECTIVES AND PROPOSED SYSTEM

The primary aim of this project is to design and implement an automated system capable of extracting structured tabular information from invoice documents. A system focusing on improving the accuracy and reliability of invoice data extraction while minimizing manual work.

4.1 Objectives of the Project

The major objectives of the project are outlined below:

1. To develop a system capable of automatically detecting table regions in invoice documents.
2. To extract textual content from detected table regions using OCR techniques.
3. To analyze relationships between recognized text elements and reconstruct the logical row–column structure of invoice tables.
4. To design algorithms that can handle irregular table layouts.
5. To export extracted tabular data into structured formats such as Excel spreadsheets.
6. To provide confidence metrics that allow users to evaluate the reliability of extracted results.
7. To develop a simple and interactive user interface for uploading documents and visualizing extraction results.

4.2 Proposed System

The system is built around a step-by-step pipeline where each stage handles a specific part of the invoice processing task. Rather than treating the document as a single block of content, the pipeline breaks the work down into different stages, making the overall process easier to track, manage, test, and improve over time.

It starts with document preprocessing, where the uploaded invoice is prepared for analysis. This includes basic operations like image normalization, noise reduction, and grayscale conversion, small but important steps.

Next, a layout detection external model (YOLO) scans the uploaded document to identify the structural components, such as tables, text blocks, and headers. Instead of processing the entire page, this stage narrows the focus down to the specific regions, i.e., the structure that actually contains table data, saving time and improving accuracy.

Once the table regions are located, those sections are cropped and passed to the OCR module. The OCR engine reads the text within each region and returns not just the recognized words, but also their exact positions on the page. Those coordinates become the foundation for further processing. With the text and its spatial data gathered, the system performs structural analysis. It groups text elements based on their horizontal and vertical positions to identify where rows begin and end and the column’s alignments. This step works even when no visible grid lines are present, relying purely on spacing and geometry to understand the table's layout.

From there, the system reconstructs the logical rows of the table. This is especially useful when item descriptions wrap across multiple lines or when OCR splits content in unexpected ways. Once rows and columns are both confirmed, the data is assembled into a clean table matrix where every value sits in its correct cell.

The final step exports each detected table into a separate sheet within an Excel file, making the results immediately usable within standard business tools.

On top of this, the system calculates confidence metrics, including OCR confidence scores, cell completeness, and numerical consistency to give users a clear sense of how reliable each extraction result is. Every module in the pipeline is independently replaceable, keeping the system flexible and ready to handle new document formats as needs evolve.

5. LITERATURE SURVEY

The task of extracting structured information from document images has been widely studied in the domains of document analysis and computer vision. Over the years, researchers have proposed different techniques to detect tables, extract text, and reconstruct structured data from unstructured documents.

Early approaches to document layout analysis relied heavily on image processing techniques. Methods based on edge detection, Hough transforms, and connected component analysis were commonly used to identify table boundaries (Kieninger and Dengel, 1998). These techniques performed reasonably well for documents with clear grid lines but struggled when tables lacked visible borders or had irregular formatting.

With the advancement of deep learning, object detection models have been increasingly used for document layout analysis. Models such as Faster R-CNN (Ren et al., 2015) and YOLO (Redmon et al., 2016) treat layout elements as objects and detect them using bounding boxes. More recent work, such as DocLayNet (Pfitzmann et al., 2022), has demonstrated the effectiveness of such models in identifying complex document structures, including tables, headers, and text blocks.

Optical Character Recognition (OCR) plays a central role in extracting textual information from document images. Traditional OCR engines like Tesseract (Smith, 2007) have been widely used for this purpose. However, their performance is often limited when dealing with noisy images or varied fonts. Deep learning-based OCR systems, such as PaddleOCR (Du et al., 2020), have improved recognition accuracy by combining detection and recognition in a unified framework and supporting multi-orientation text.

Another important aspect of table extraction is the identification of row and column structures. Early methods depended on detecting explicit table lines to segment rows and columns. However, as noted by Shafait et al. (2010), such approaches are not suitable for borderless tables. As a result, text alignment-based methods have gained attention, where spatial relationships between words are used to infer structure.

Clustering techniques have been explored to identify column alignments in such cases. Algorithms like K-Means and DBSCAN (Ester et al., 1996) group text elements based on their spatial proximity. DBSCAN is particularly useful in this context because it does not require the number of clusters to be predefined and can effectively handle noise in the data.

Overall, the literature reflects a gradual transition from rule-based and image-processing approaches to more flexible and data-driven techniques that combine layout detection, OCR, and spatial analysis.

6. LITERATURE REVIEW

Despite the advancements in document analysis, extracting structured tables from invoices remains a challenging problem due to the variability in document layouts and formatting styles.

OCR-based approaches, although effective for extracting textual content, lack the ability to capture structural relationships between text elements. As highlighted by Smith (2007), OCR systems are primarily designed for text recognition and do not provide information about how words are organized within a document. This limitation makes it difficult to reconstruct tables directly from OCR output.

Line-based table detection methods attempt to address this issue by identifying horizontal and vertical lines in documents. While these methods work well for structured tables with clear borders, they fail when applied to borderless or partially structured tables, which are common in real-world invoices (Shafait et al., 2010).

Rule-based systems have also been proposed, where heuristics are used to identify rows and columns based on predefined patterns. However, these systems are often tailored to specific document formats and lack the flexibility to handle variations in layout. As noted by Kieninger and Dengel (1998), such approaches require extensive manual tuning and do not generalize well across different datasets.

The introduction of deep learning models for layout detection has improved the ability to identify table regions accurately. However, these models focus primarily on detecting the presence of tables rather than reconstructing their internal structure (Pfitzmann et al., 2022). As a result, additional processing is required to extract meaningful tabular data.

Clustering-based approaches have been explored to infer column structures by grouping text elements based on spatial alignment. DBSCAN (Ester et al., 1996) has been particularly effective in handling irregular data distributions. However, challenges remain in handling multi-line cells, overlapping text, and inconsistencies in alignment.

From the existing literature, it can be observed that most approaches either focus on text extraction or structural detection, but not both in a unified manner. Furthermore, many methods rely on assumptions about table formatting, which limits their applicability in real-world scenarios.

These observations highlight the need for a more general approach that can handle diverse document layouts without relying on strict rules or predefined templates.

7. SCOPE OF PROJECT AND EXPECTED OUTCOMES

The primary focus of this project is to design and implement an automated system capable of extracting structured tabular information from invoice documents. The system aims to handle a variety of invoice layouts and provide reliable extraction results that can be easily integrated into digital workflows.

7.1 Scope of the Project

The scope of this project includes the following functionalities:

1. Detection of table regions within invoice documents using document layout analysis techniques.
2. Extraction of textual content from detected table regions using optical character recognition.
3. Identification of row boundaries and column alignments based on spatial relationships between recognized text elements.
4. Reconstruction of logical rows from fragmented OCR outputs, especially for item descriptions that span multiple lines.
5. Generation of structured table representations from the reconstructed rows and columns.
6. Export of extracted tables into spreadsheet formats such as Microsoft Excel.
7. Visualization of extraction results through a user interface for demonstration and evaluation.

The system is designed to process invoice documents provided in image formats such as JPG or PNG. Additionally, PDF documents containing a limited number of pages can also be processed by converting them into images before analysis. Although the system is designed to handle a variety of invoice layouts, it primarily focuses on tables containing itemized product information like invoices.

7.2 Expected Outcomes

The expected outcomes of the project include the development of a functional prototype capable of automatically extracting tabular data from invoice documents. The system should be able to detect table structures, recognize textual content, and reconstruct structured tables with reasonable accuracy.

Another expected outcome is the generation of structured spreadsheet files that contain the extracted data. These spreadsheets allow users to review and utilize the extracted information in other applications such as accounting software or data analysis tools.

The project also aims to provide confidence metrics that indicate the reliability of the extraction results. These metrics help users assess whether the extracted data is suitable for direct use or requires manual verification and correction.

From an academic perspective, the project demonstrates the application of document analysis techniques, machine learning models, and clustering algorithms to solve real-world information extraction problems. In summary, the project aims to deliver a practical solution for invoice table extraction.

8. TOOLS AND TECHNOLOGIES

The development of the system involves several software tools and technologies that support document analysis, optical character recognition, data processing, and user interface development. Each component plays an important role in enabling the system to perform accurate table extraction from invoice documents.

8.1 Technologies

Below are the tools and technology used in project development

- **Code Editor**
  - Visual Studio Code (VS Code)

- **Python Programming Language (Python 3.10.11)**
  - Python is the primary programming language used for implementing the system.
  - Python provides extensive support for machine learning, image processing, and data analysis through a large ecosystem of libraries and frameworks.
  - Its readability and flexibility make it well suited for rapid development of complex pipelines.

- **OpenCV**
  - OpenCV is an open-source computer vision library used for image processing tasks within the system.
  - It provides functions for image preprocessing operations such as grayscale conversion, thresholding, noise reduction, and edge detection.
  - These operations help improve the quality of images before they are processed by OCR or layout detection models.

- **YOLO-Based Layout Detection Model**
  - The system uses a layout detection model based on the YOLO (You Only Look Once) architecture to identify structural elements within documents.
  - YOLO models are widely used in object detection tasks because they provide high detection accuracy while maintaining fast inference speeds.
  - In this project, the layout detection model is used to locate table regions within invoice images.

- **PaddleOCR**
  - PaddleOCR is an open-source optical character recognition framework designed for efficient and accurate text recognition.
  - It integrates multiple components, including text detection, text recognition, and text orientation classification.
  - PaddleOCR provides pretrained models capable of recognizing printed text from document images with high accuracy.

- **Streamlit**
  - Streamlit is used to develop the graphical user interface of the system.
  - It provides a simple framework for building interactive web applications using Python.
  - The interface allows users to upload invoice images, visualize extraction results, inspect detected tables, and download exported spreadsheets.

- **OpenPyXL**.
  - The system uses OpenPyXL to export the reconstructed tables into structured Excel spreadsheets, enabling users to easily access and analyze the extracted data.

- **PDF2Image**
  - This allows the system to support invoice documents that are provided in PDF format.

- **NumPy**
  - NumPy is used for numerical computations and array manipulation.

Together, these tools and technologies enable the development of a complete pipeline for automated invoice table extraction. The combination of machine learning models, image processing techniques, and clustering algorithms provides a robust foundation for extracting structured information from diverse document layouts.

8.2 System Requirements

The successful development and execution of the DocuStruct system require a suitable hardware and software environment. The requirements are categorized into development requirements and usage (deployment) requirements.

8.2.1 Development Requirements

The development environment must support machine learning libraries, image processing, and web-based interface frameworks.

Hardware Requirements:

- Processor: Intel i5/AMD Ryzen 5 or equivalent
- RAM: Minimum 8 GB (16 GB recommended for smoother performance)
- Storage: At least 10 GB free space
- GPU: Optional (CPU-based execution is supported, but GPU improves performance)

Software Requirements:

- Operating System: Windows 10/11, Linux, or macOS
- Programming Language: Python 3.10.11 or higher
- Development Environment: VS Code / PyCharm
- Required Libraries: As mentioned earlier in section 8.1

8.2.2 Network Requirements

- Internet connection is required only during initial setup for downloading models and dependencies
- The system can run offline once all components are installed

8.3 Deployment

DocuStruct is deployed using Hugging Face Spaces, model and tokens with a Docker-based setup, enabling seamless hosting of the Streamlit application.

Deployment Steps

1. Model Upload
   YOLO layout model was uploaded to Hugging Face Model Hub
   Model is loaded dynamically during runtime
   https://huggingface.co/shivam2k2/docustruct-yolo

2. Configure README (YAML)

3. Docker Setup

```
FROM python:3.10-slim

WORKDIR /app

# Install system dependencies
RUN apt-get update && apt-get install -y \
    libgl1 \
    libglib2.0-0 \
    tesseract-ocr \
    poppler-utils \
    && rm -rf /var/lib/apt/lists/*

# Copy project files
COPY . .

# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt

# Expose required port
EXPOSE 7860

# Run Streamlit app with ALL required configs
CMD ["streamlit", "run", "app.py", "--server.port=7860",
"--server.address=0.0.0.0", "--server.enableCORS=false",
"--server.enableXsrfProtection=false", "--server.maxUploadSize=200"]
```

4. Create Hugging Face Space
    - Platform: Hugging Face Spaces
    - SDK: Docker + Streamlit
    - Hardware: CPU (Free Tier)

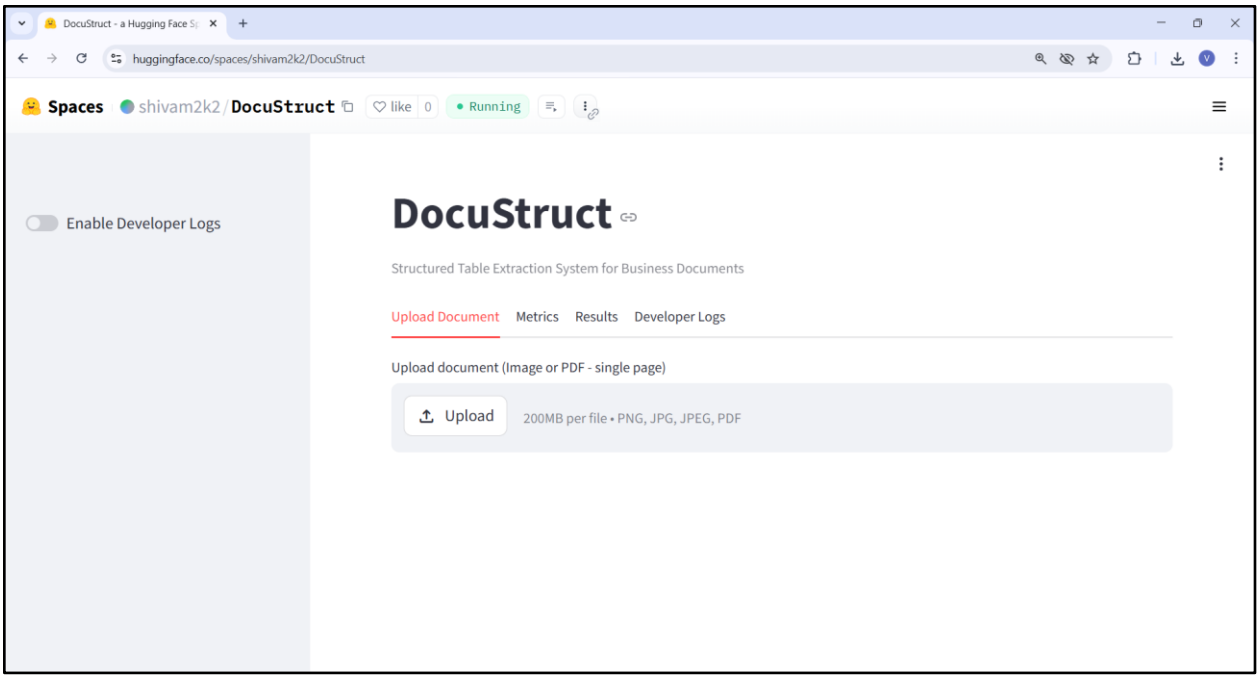
5. Push Code to Space

   Used Hugging Face access token to push clean repository to HF Space

6. Automatic Build & Launch

   Hugging Face:
    - Builds Docker container
    - Installs dependencies
    - Launches application

Deployed link: https://huggingface.co/spaces/shivam2k2/DocuStruct



9. PROJECT DESIGN

The design of the system focuses on building a robust pipeline capable of extracting structured tabular information from invoice documents. The system must be able to handle diverse document structures without relying on rigid template-based rules. To achieve this, the project adopts a modular design approach in which the overall extraction process is divided into multiple independent components.

Each component of the system performs a specific task and passes its output to the next stage of the pipeline. This modular architecture provides several advantages, including flexibility, maintainability, and scalability. Individual components can be improved or replaced without affecting the overall functionality of the system.

The complete pipeline of the proposed system consists of the following major stages: (for details refer section 9.2)

1. Image Input and Preprocessing
2. Document Layout Detection
3. Table Region Extraction
4. Optical Character Recognition (OCR)
5. Row Detection
6. Column Detection
7. Logical Row Reconstruction
8. Table Matrix Construction
9. Data Export and Confidence Analysis

1. **Image Input and Preprocessing**

The first step of the system involves accepting invoice documents as input. The system supports both image files and PDF documents. If the input document is in PDF format, it is converted into images using a PDF-to-image conversion utility. Once the document is available as an image, preprocessing operations are performed to improve its quality.

Preprocessing may include grayscale conversion, noise removal, and normalization. These steps help enhance the visibility of textual content and reduce artifacts that might interfere with OCR accuracy.

2. **Document Layout Detection**

After preprocessing, the system performs layout analysis to identify structural elements within the document. A layout detection model is used to detect regions such as tables, text blocks, and headers. The goal of this stage is to locate the table regions within the invoice.

Detecting tables at an early stage allows the system to focus subsequent processing only on relevant areas of the document. This improves both efficiency and accuracy.

3. **Table Region Extraction**

Once table regions are identified, the system extracts the corresponding portions of the document image. Each detected table region is treated as an independent processing unit. This approach allows the system to handle documents containing multiple tables.

4. **Optical Character Recognition**

The extracted table regions are then passed to the OCR module. The OCR system detects individual text elements within the image and recognizes the characters present in each detected text region. In addition to textual content, the OCR module also provides bounding box coordinates that indicate the spatial location of each recognized word.

These spatial coordinates are essential for reconstructing the structural relationships between different text elements.

5. **Row Detection**

After OCR processing, the system analyzes the vertical alignment of text elements to determine potential row boundaries. Words that share similar vertical positions are grouped together to form row segments.

Row detection is particularly important for invoice tables because item descriptions often span multiple lines. The system must correctly identify which text segments belong to the same logical row.

6. **Column Detection**

Column detection is performed by analyzing the horizontal positions of OCR-detected words. Words that share similar horizontal coordinates are likely to belong to the same column.

Clustering algorithms are used to group text elements into column clusters. This approach works even when tables do not have visible borders separating columns.

7. **Logical Row Reconstruction**

In many cases, the OCR system may split a single table row into multiple segments due to multi-line descriptions or irregular spacing. The logical row reconstruction module merges these segments based on spatial proximity and column alignment. This step ensures that each logical row of the table corresponds to a single item entry.

8. **Table Matrix Construction**

Once rows and columns have been identified, the system constructs a table matrix where each cell corresponds to a specific row and column position. Words belonging to the same cell are combined to form the final cell content.

9. **Data Export and Confidence Analysis**

The final stage of the system involves exporting the reconstructed table into a structured format such as an Excel spreadsheet. Each detected table can be exported to a separate sheet within the same workbook.

In addition to exporting the data, the system also calculates confidence metrics based on OCR confidence scores and structural completeness of the reconstructed table. These metrics help users assess the reliability of the extracted results.

The modular design of the system ensures that each stage of the pipeline can be independently improved or replaced in future iterations of the project.

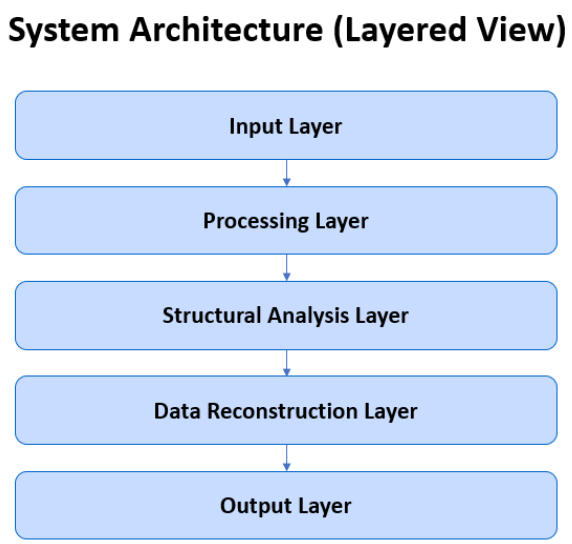
9.1 System/Project Design

System Architecture

The architecture of the proposed invoice table extraction system follows a layered pipeline structure. Each layer of the architecture performs a specific task and communicates with adjacent layers through well-defined data interfaces.

The system architecture can be divided into the following layers:

- Input Layer
- Processing Layer
- Structural Analysis Layer
- Data Reconstruction Layer
- Output Layer



9.1.1 Input Layer

The input layer is responsible for accepting invoice documents from the user. The system supports image formats such as JPG and PNG, as well as PDF documents containing a limited number of pages. When a PDF document is uploaded, it is first converted into images to enable further processing.

9.1.2 Processing Layer

The processing layer performs image preprocessing and document layout analysis. Image preprocessing enhances the quality of the document image, making it easier for subsequent modules to detect text and structural elements.

The layout detection module analyzes the document to locate tables and other structural components. This stage isolates the table regions that contain the relevant data to be extracted.

9.1.3 Structural Analysis Layer

Once the table regions have been identified, the OCR engine extracts textual content from the image. The OCR system returns both the recognized text and the bounding box coordinates for each detected word.

These coordinates are used by the structural analysis modules to determine row and column relationships between words. Row detection algorithms analyze vertical alignment, while column detection algorithms analyze horizontal alignment.

9.1.4 Data Reconstruction Layer

After rows and columns have been detected, the system reconstructs the logical structure of the table. This involves merging fragmented text segments into complete rows and assigning each word to the appropriate table cell.

The reconstructed data is stored in a two-dimensional table matrix where each entry represents a specific cell within the table.

9.1.5 Output Layer

The output layer converts the structured table matrix into a user-friendly format. The extracted data is exported into an Excel spreadsheet, where each detected table is stored in a separate sheet.

The system also generates confidence metrics that help users evaluate the reliability of the extraction results.

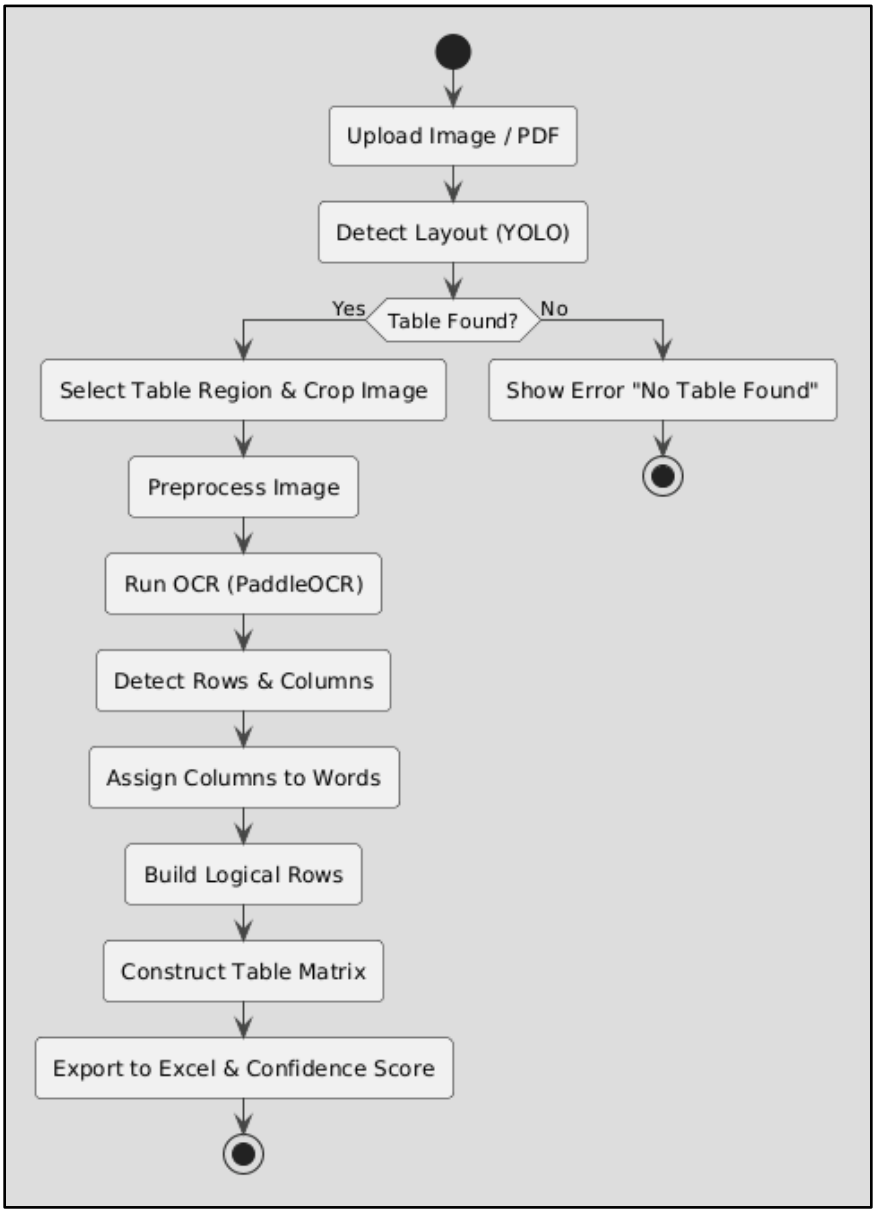
This layered architecture ensures clear separation of responsibilities between system components. It also allows developers to modify or upgrade individual modules without affecting the entire pipeline.

9.2 UML Diagrams

To better understand the workflow and internal structure of the proposed system, several UML diagrams are used to illustrate the interactions between different components of the system.

9.2.1 Activity Diagram

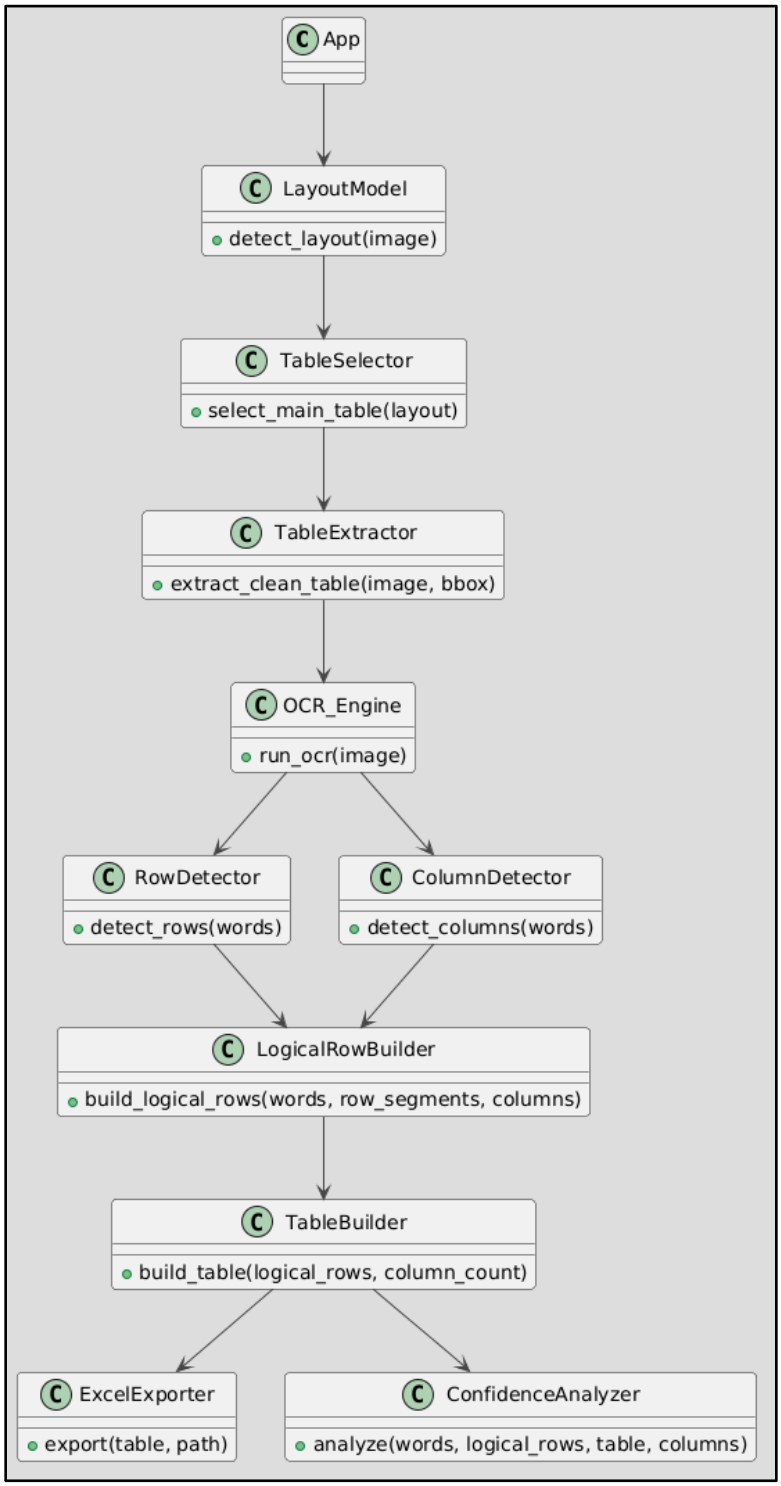
The activity diagram represents the sequence of operations performed during the invoice processing pipeline.



This diagram represents the complete workflow of the system. The process begins with uploading an invoice image or PDF file. The layout detection module identifies table regions in the document. If a table is detected, the system proceeds with extracting and processing the table. OCR is applied to extract textual data, followed by row and column detection to reconstruct the table structure. The final structured table is exported to Excel, and confidence metrics are generated. If no table is detected, the system terminates with an appropriate message.

9.2.2 Class Diagram

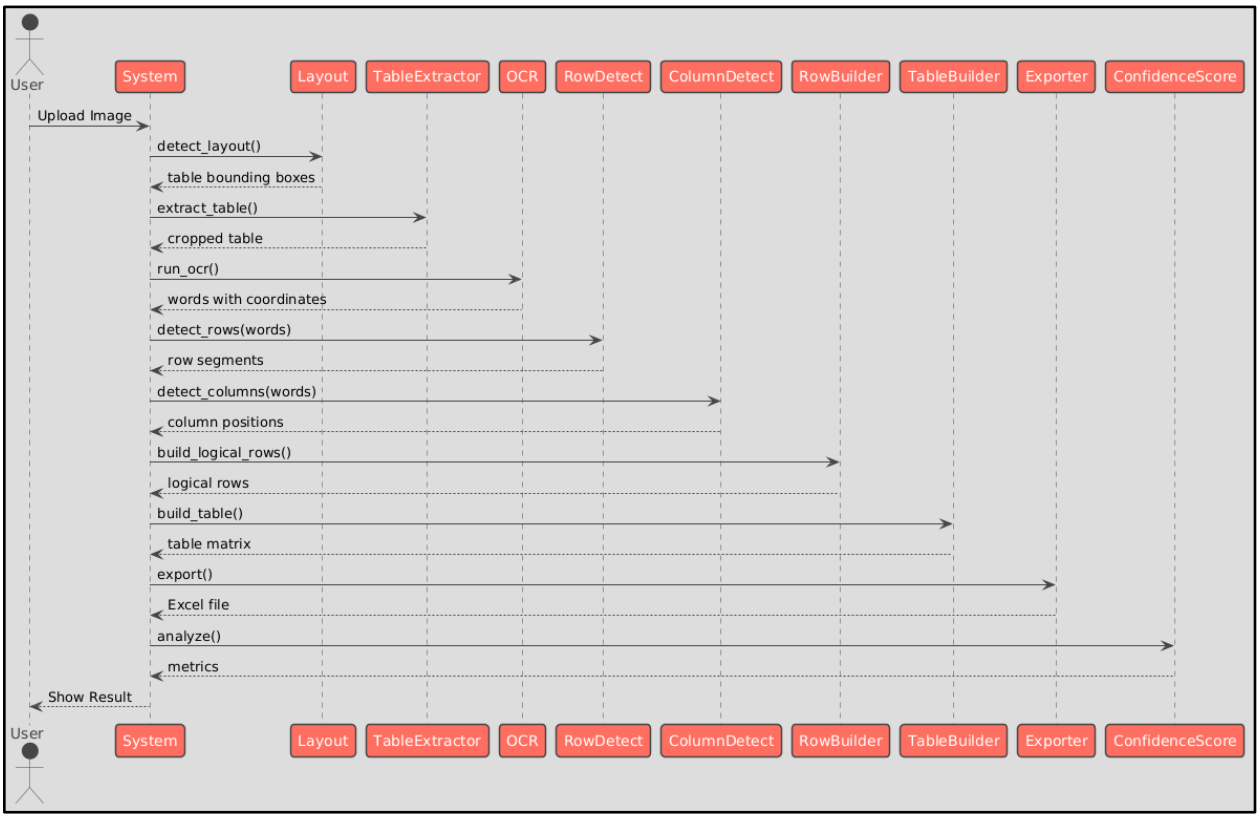
The class diagram represents the core software components of the system and the relationships between them. Each module of the pipeline is implemented as a separate class responsible for a specific task.



The class diagram represents the modular structure of the system. Each class is responsible for a specific function, such as OCR processing, row detection, or table construction. The relationships between classes show how data flows through the system. This modular design improves maintainability and scalability.

9.2.3 Sequence Diagram

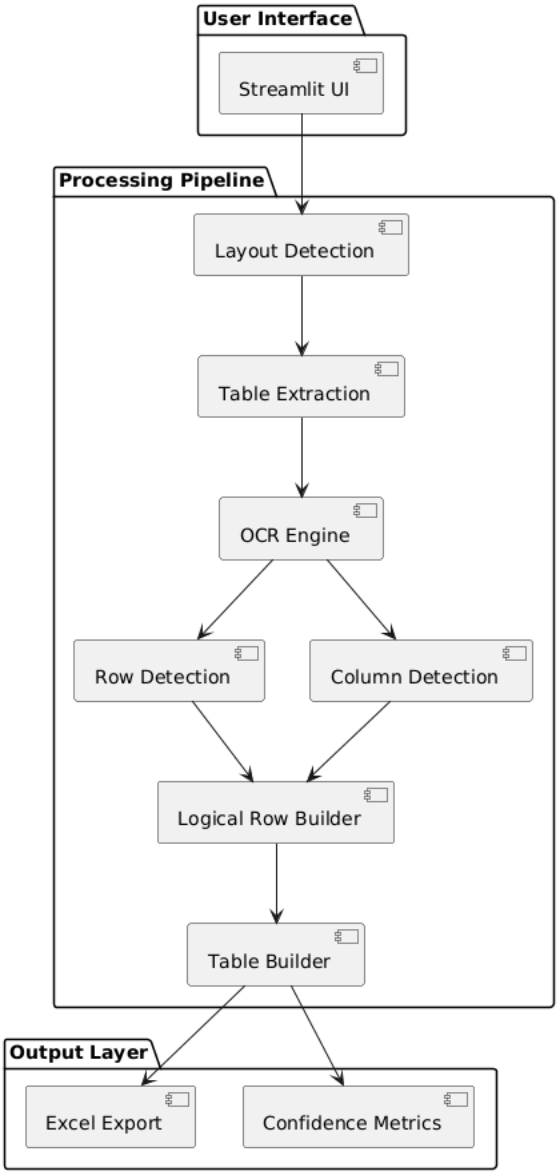
The sequence diagram illustrates the interaction between system components during the extraction process.



It illustrates the interaction between different modules of the system. The user uploads an image, and the system processes it step-by-step. Each module performs a specific task, such as layout detection, OCR, row detection, and table construction. The output from each module serves as input for the next module, ensuring a structured and modular workflow.

9.2.4 Component Diagram

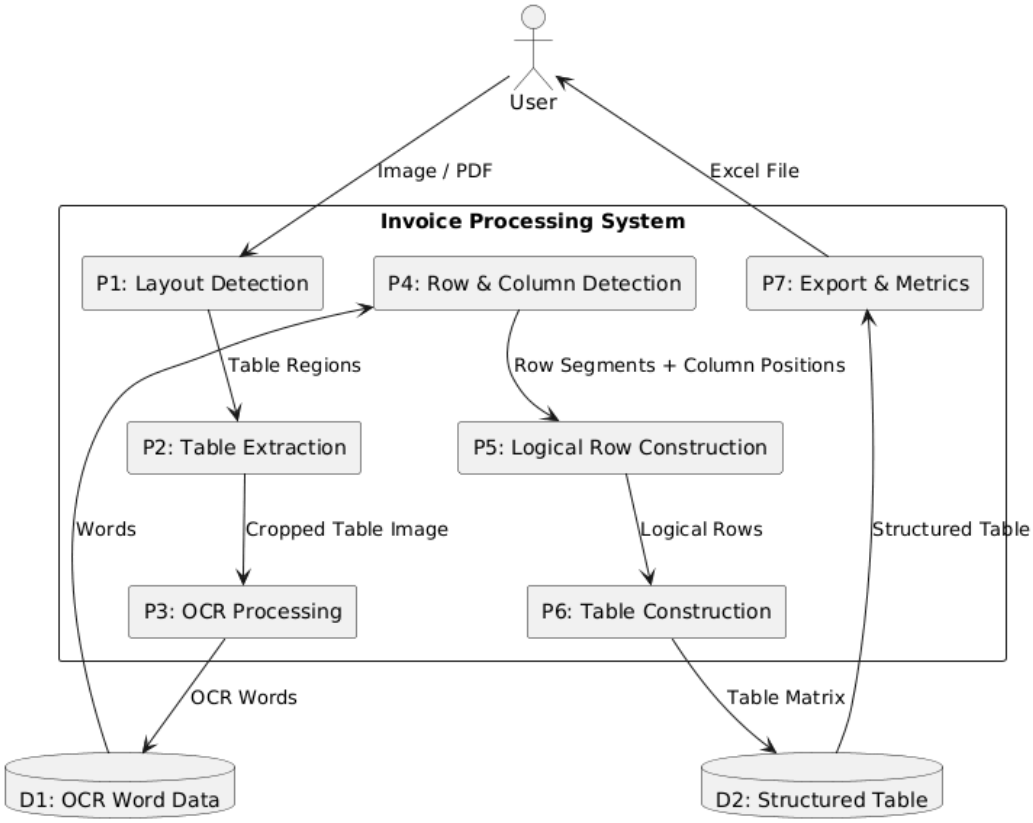
The component diagram represents the high-level architecture of the system and the dependencies between different software modules.



This diagram represents the high-level architecture of the system. It is divided into three main components: user interface, processing pipeline, and output layer. Each component performs a distinct function, and data flows sequentially from input to output.

9.2.5 Data Flow Diagram

The Data Flow Diagram (DFD) illustrates how data is processed within the system.



The user provides an invoice image or PDF as input. The system first performs layout detection to identify table regions. These regions are extracted and passed to the OCR module, which generates word-level textual data. The extracted words are stored temporarily and used for detecting rows and columns. These structural elements are combined to form logical rows, which are then used to construct a structured table. The final table is stored and exported as an Excel file along with confidence metrics. The DFD highlights the transformation of raw input data into structured output through multiple processing stages.

10. PROJECT DEVELOPMENT AND IMPLEMENTATION

The development of the proposed invoice table extraction system followed a modular and iterative approach. Instead of building the entire system at once, individual modules were developed, tested, and integrated sequentially. This approach allowed for easier debugging, performance evaluation, and improvement of individual components.

The system was implemented using Python due to its extensive ecosystem of libraries for machine learning, computer vision, and document processing. The development process focused on creating reusable modules that perform specific tasks within the extraction pipeline.

The development process can be broadly divided into the following stages as we have already discussed in Project design section:

1. Image Processing and Input Handling
2. Document Layout Detection
3. Table Extraction
4. Optical Character Recognition
5. Row Detection
6. Column Detection
7. Logical Row Reconstruction
8. Table Construction
9. Confidence Analysis
10. Data Export
11. User Interface Development

10.1 Project Development Steps with Software Code

The development of the invoice table extraction system involved implementing several software modules that together form the complete processing pipeline. Each module performs a specific task and communicates with other modules through structured data formats.

Step 1: Loading and Preprocessing the Image

The first step involves loading the invoice image and preparing it for further processing.

```python
import cv2

def load_image(path):
    image = cv2.imread(path)
    return image

processed = preprocess_image(image)
# next module will get this processed image and perform further analysis
```

This function loads the image using OpenCV and returns it for further processing.

Step 2: Detecting Document Layout

The layout detection module identifies table regions within the invoice.

```python
from ultralytics import YOLO

model = YOLO("layout_model.pt") # model path

def detect_layout(image):
    results = model(image)
    return results # return the results of the layout detection
```

The model analyzes the image and returns bounding boxes corresponding to detected document components.

Step 3: Extracting Table Regions

Once table regions are detected, the corresponding portions of the image are extracted.

```python
def extract_table(image, bbox):
    # Extract the table from the image using the bounding box coordinates
    x1, y1, x2, y2 = bbox
    return image[y1:y2, x1:x2] # table region extracted from the image
```

This function crops the table region from the invoice image.

Step 4: Performing OCR

The OCR module extracts text from the table region.

```python
from paddleocr import PaddleOCR

ocr = PaddleOCR()

def run_ocr(image):
    result = ocr.ocr(image)
    return result # returns recognized text along with  box coordinates
```

The OCR engine returns recognized text along with bounding box coordinates.

Step 5: Detecting Rows

Row detection groups words based on their vertical alignment.

```python
def detect_rows(words):
    rows = []
    """
    logic for grouping words by y-coordinate
    """
    return rows
```

Step 6: Detecting Columns

Column detection clusters words based on their horizontal alignment.

```python
from sklearn.cluster import DBSCAN

def detect_columns(words):
    """
    clustering logic
    """
    return columns
```

Step 7: Constructing the Table Matrix

After detecting rows and columns, the table matrix is constructed.

```python
rows = detect_rows(words)
columns = detect_columns(words)

def build_table(rows, columns):
    table = []
    """
    assign words to cells
    """
    return table
```

Step 8: Exporting to Excel

The final structured table is exported to Excel format.

```python
from openpyxl import Workbook

def export_to_excel(table, path):
    wb = Workbook()
    ws = wb.active

    for row in table:
        ws.append(row)

    wb.save(path)
```

These development steps demonstrate how the system integrates multiple modules to transform raw invoice images into structured spreadsheet data.

UI(Frontend - Streamlit)

```
import os
import streamlit as st # type: ignore
import numpy as np
import cv2
from pdf2image import convert_from_bytes
from app_entry import run_application
from utlis.logger import AppLogger

logger = AppLogger()

# Initialize logs
if "logs" not in st.session_state:
    st.session_state["logs"] = []

APP_TITLE = "DocuStruct"
st.set_page_config(page_title=APP_TITLE, layout="wide")

st.markdown("""...

# SIDEBAR
developer_mode = st.sidebar.toggle("Enable Developer Logs")

# HEADER
st.title(APP_TITLE)
st.caption("Structured Table Extraction System for Business Documents")

# TABS
tab1, tab2, tab3, tab4 = st.tabs(["Upload Document", "Metrics", "Results", "Developer Logs"])

# TAB 1 - UPLOAD
with tab1:...

# TAB 2 - METRICS
with tab2:...

# TAB 3 - RESULTS
with tab3:...

# TAB 4 - DEVELOPER LOGS
with tab4:...
```

Find the entire codebase using below link

GitHub: https://github.com/shivamvishwakarma2k2/invoice_table_extractor

10.2 Project Testing

Testing is an essential phase of software development that ensures the correctness, reliability, and performance of the developed system. The proposed invoice table extraction system was tested using multiple invoice images with different layouts and formatting styles. The testing process focused on verifying the functionality of each module in the pipeline as well as evaluating the overall performance of the integrated system.

The primary objectives of testing were :

1. To verify that each module of the system performs its intended function correctly.
2. To ensure that the system can process invoices with varying layouts.
3. To evaluate the accuracy of table detection and text extraction.
4. To validate the correctness of reconstructed table structures.
5. To confirm that the final structured data is exported correctly.

The testing process involved multiple levels, including unit testing, integration testing, and system testing.

10.2.1 Unit Testing

Unit testing focuses on verifying the functionality of individual modules independently. Each module of the pipeline was tested using controlled input data to ensure correct behavior.

Unit tests include:

- Testing the image preprocessing module to verify that images are correctly loaded and normalized.
- Testing the layout detection module to confirm that table regions are accurately identified.
- Testing the OCR module to ensure correct text recognition from table images.
- Testing row and column detection algorithms to verify that spatial clustering works correctly.

10.2.2 Integration Testing

Integration testing was performed after individual modules were verified. This stage involved connecting different modules of the pipeline and testing their interaction.

- Layout detection output was passed to the table extraction module.
- Extracted table regions were passed to the OCR engine.
- OCR results were used as input for row and column detection modules.

This testing ensured that data was correctly transferred between modules and that the pipeline executed sequentially without errors.

10.2.3 Functional Testing

Functional testing verifies whether the system meets the specified functional requirements. The following test cases were used during evaluation.

Test Case Table

ID	Test Description	Input	Expected Output	Actual Output	Result
TC01	Upload Image File	Valid JPG/PNG image	Image successfully uploaded and preview	Image uploaded and previewed	Pass
TC02	Upload PDF File	Valid PDF	PDF converted to images and preview	PDF processed and previewed	Pass
TC03	Invalid File Upload	Unsupported file (e.g., .txt)	Error message displayed	Error message shown	Pass
TC04	Layout Detection	Invoice image with table	Table region detected	Table detected correctly	Pass
TC05	No Table Detection	Image without table	Error: "No table detected"	Error message displayed	Pass
TC06	Table Extraction	Detected table region	Table cropped correctly	Table extracted correctly	Pass
TC07	OCR Processing	Table image	Text extracted with bounding boxes	OCR output generated	Pass
TC08	Row Detection	OCR words	Rows correctly grouped	Rows detected correctly	Pass
TC09	Column Detection	OCR words	Columns correctly identified	Columns detected correctly	Pass

TC10	Logical Row Construction	Rows + Columns	Rows merged correctly	Logical rows constructed	Pass
TC11	Table Matrix Construction	Logical rows + columns	Structured table generated	Table matrix created	Pass
TC12	Excel Export	Structured table	Excel file generated with data	Excel file generated	Pass
TC13	Confidence Metrics Calculation	Extracted data	Metrics calculated and displayed	Metrics displayed correctly	Pass
TC14	Developer Mode Toggle	Enable developer mode	logs displayed	logs shown	Pass

10.2.4 System Testing

System testing evaluates the complete system using real invoice documents. Various invoices with different structures were used to test the robustness of the system.

The system was tested on invoices with:

- Grid-based tables - Reliable
- Borderless tables - Not reliable
- Multi-line-item descriptions - Reliable
- Multiple tables within a document - Reliable

The results were analyzed to determine the accuracy and reliability of the extraction process.

10.2.5 Performance Testing

Performance testing was conducted to evaluate the processing time required for extracting tables from invoice images. The processing time depends on factors such as image resolution and table complexity.

On average, the system processed a single invoice image within a few seconds on a standard computer system.

10.2.6 Error Handling Testing

The system was also tested for handling error scenarios such as:

- Uploading an image without tables
- Uploading unsupported file formats
- Processing low-quality images

Appropriate error messages were displayed to guide the user in such cases.

Overall, the testing results indicate that the developed system performs reliably across a variety of invoice formats.

11. RESULT AND CONCLUSION

The developed system was evaluated using multiple invoice documents containing tabular information. The evaluation focused on measuring the accuracy of table detection, text extraction, and table reconstruction.

Table Detection Results

The layout detection model successfully identified table regions in most invoice documents. The use of a layout detection model enabled the system to focus processing only on relevant regions, improving overall efficiency.

OCR Results

The OCR module demonstrated high accuracy in recognizing printed text from invoice tables. Most text elements such as item descriptions, quantities, and price values were recognized correctly.

However, minor recognition errors were occasionally observed in cases where:

- The image quality was poor
- Text was partially obscured
- Fonts were unusual or stylized

Row and Column Detection

The row detection algorithm successfully grouped words based on vertical alignment, while the column detection module identified column clusters using spatial clustering techniques.

Logical Row Reconstruction

Multi-line-item descriptions were correctly merged into single logical rows. This ensured that each item entry appeared as a complete record in the final structured table.

Excel Export

The final table matrix was successfully exported into Excel format. When multiple tables were detected in a document, each table was exported into a separate sheet within the same workbook.

Confidence Analysis

The system generated extraction metrics including OCR confidence scores and structural completeness ratios. These metrics helped evaluate the reliability of the extracted data.

Overall Performance

The overall extraction pipeline demonstrated strong performance across various invoice formats. The modular architecture allowed the system to handle different table layouts while maintaining reliable extraction results.

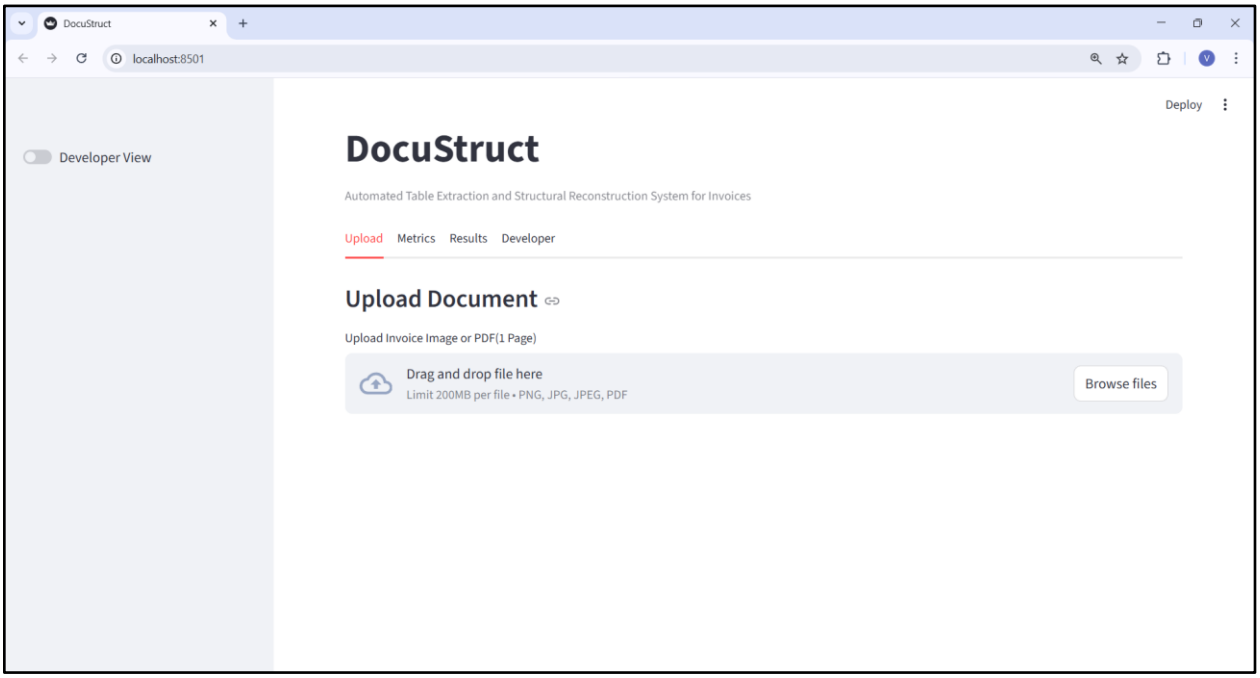
The results indicate that combining document layout detection with spatial analysis of OCR outputs is an effective approach for automated invoice table extraction.

11.1 Software Result Screenshots

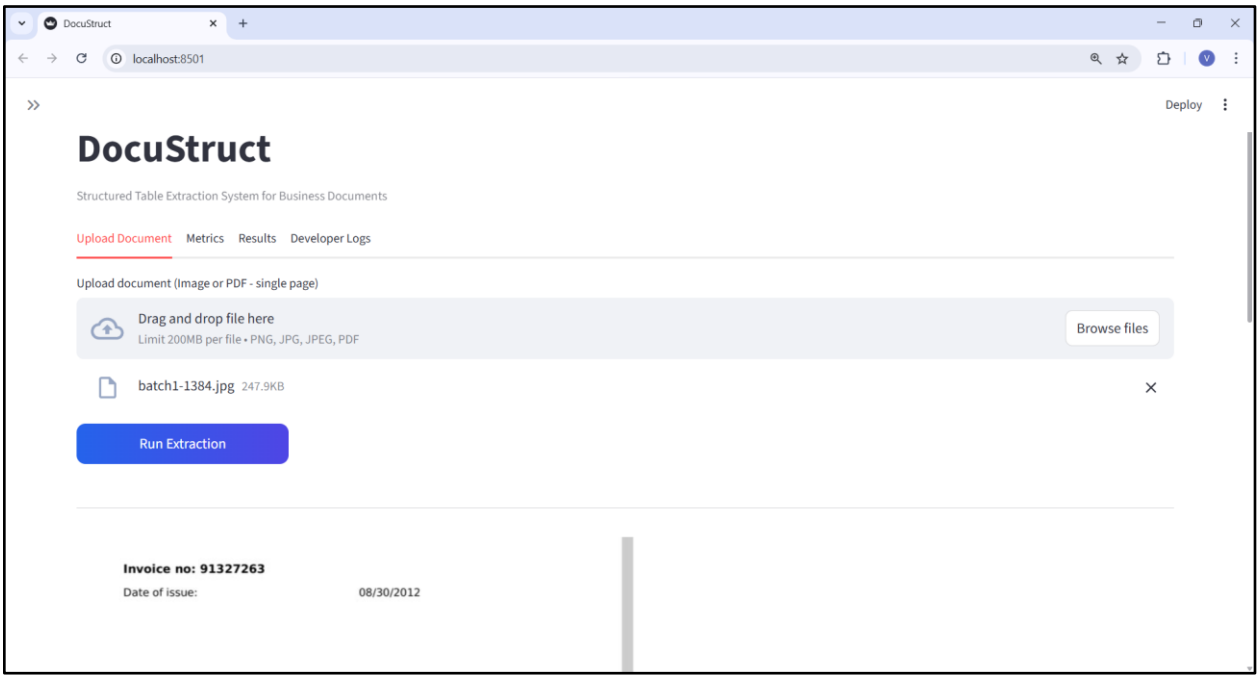
This section presents screenshots of the developed system demonstrating different stages of the invoice table extraction process.

Screenshot 1: Home Interface

The home interface of the application allows users to upload invoice images or PDF documents for processing.

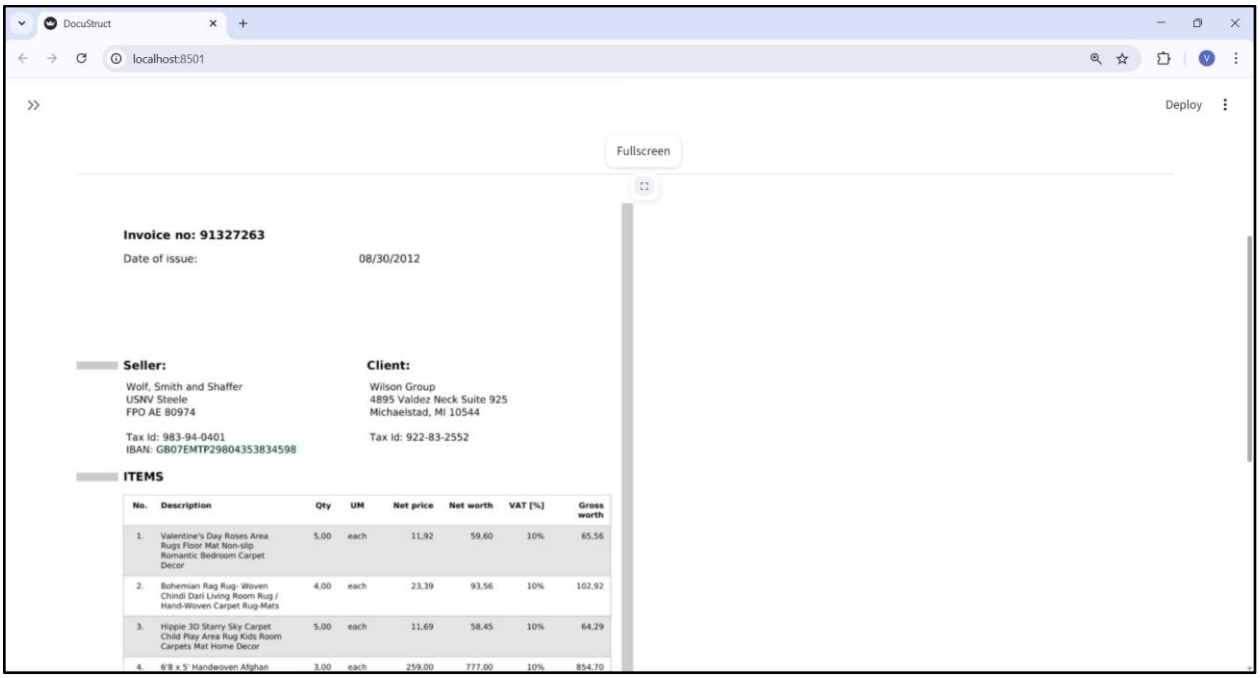


After Uploading and file

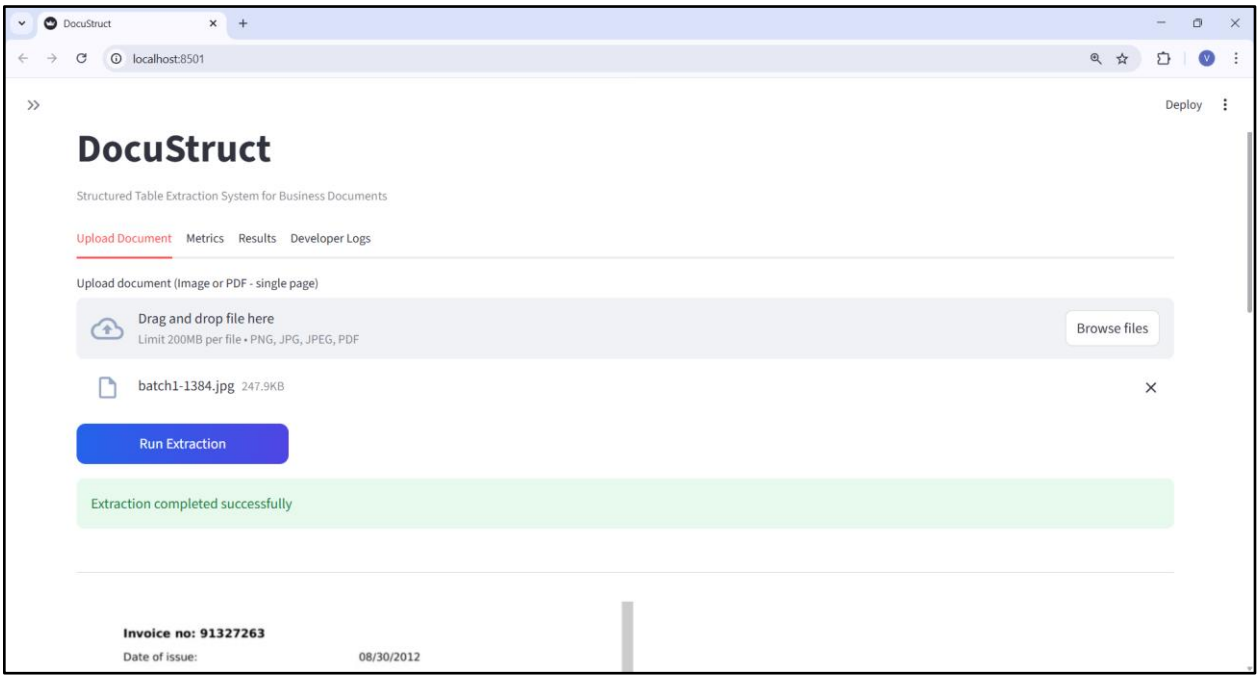


Screenshot 2: Invoice Image Preview

After uploading the invoice, the system displays a preview of the document before extraction begins.

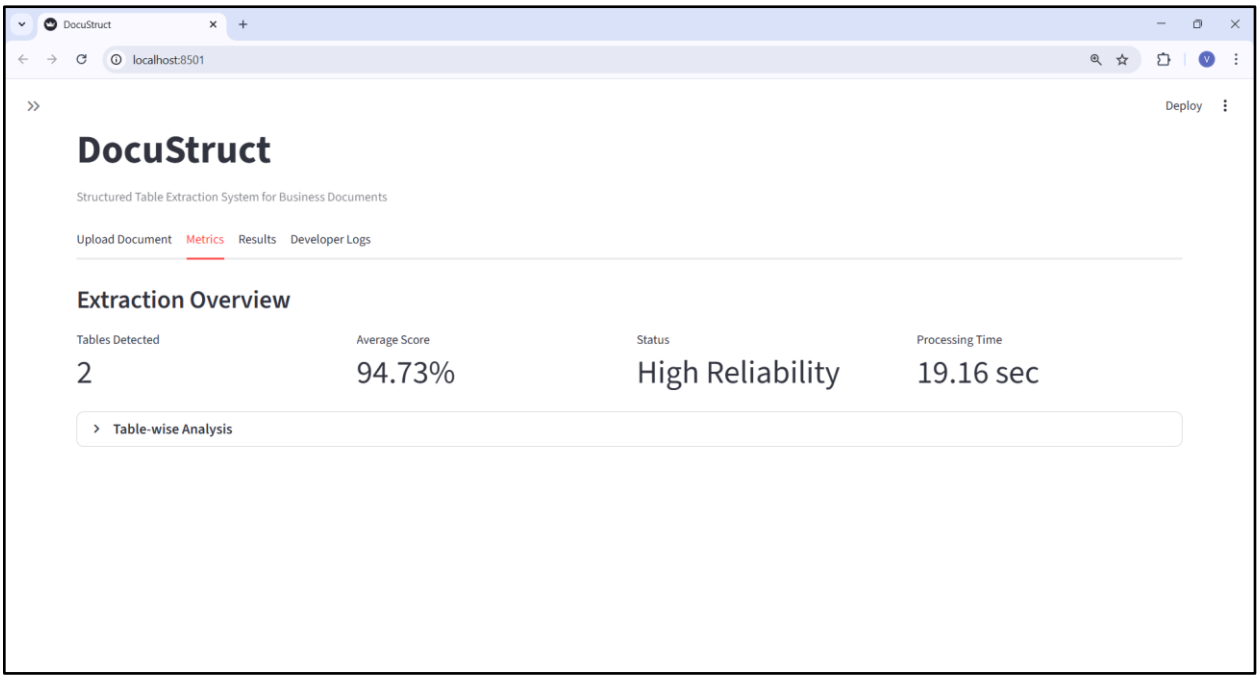


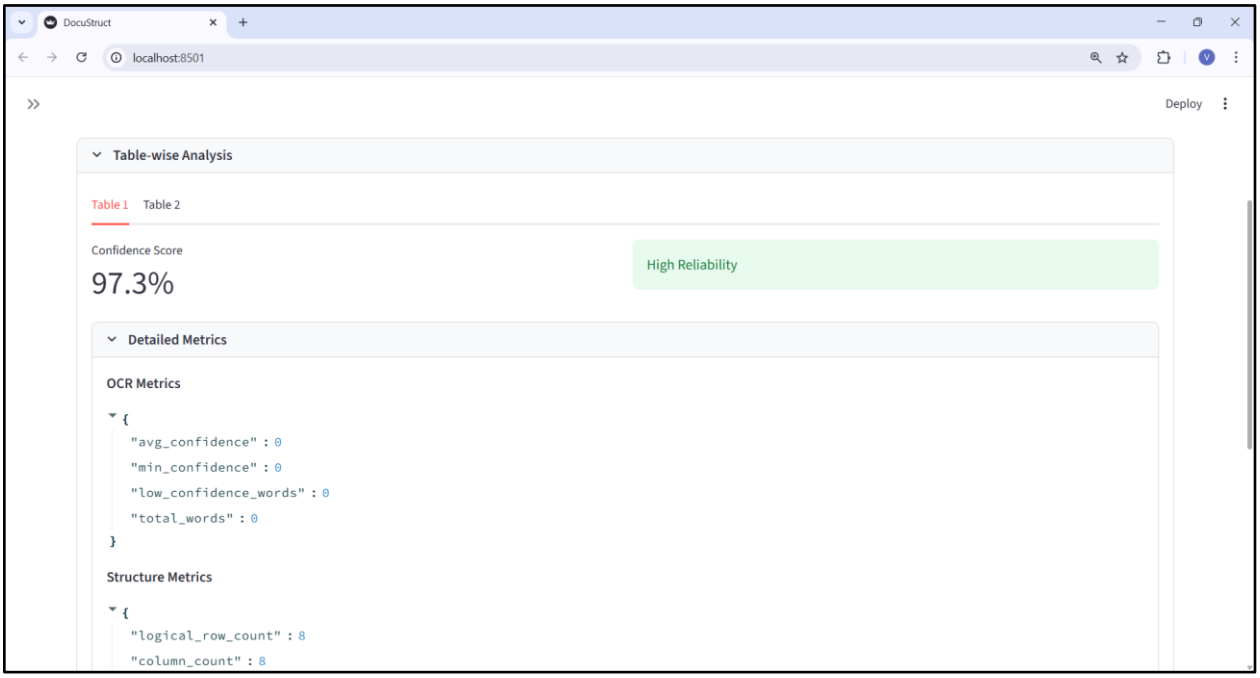
Screenshot 3: Table Extraction completed



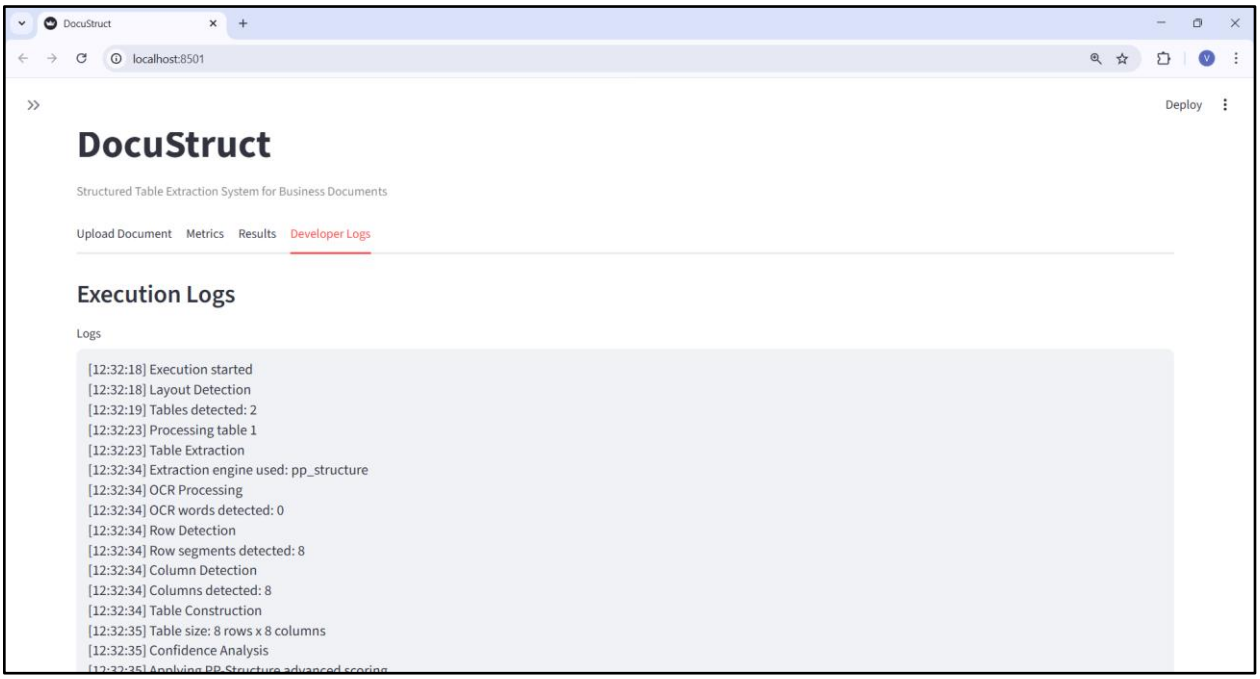
Screenshot 4: Confidence Score Visualization

The system displays the confidence score for user understanding should it be fully reliable or not.



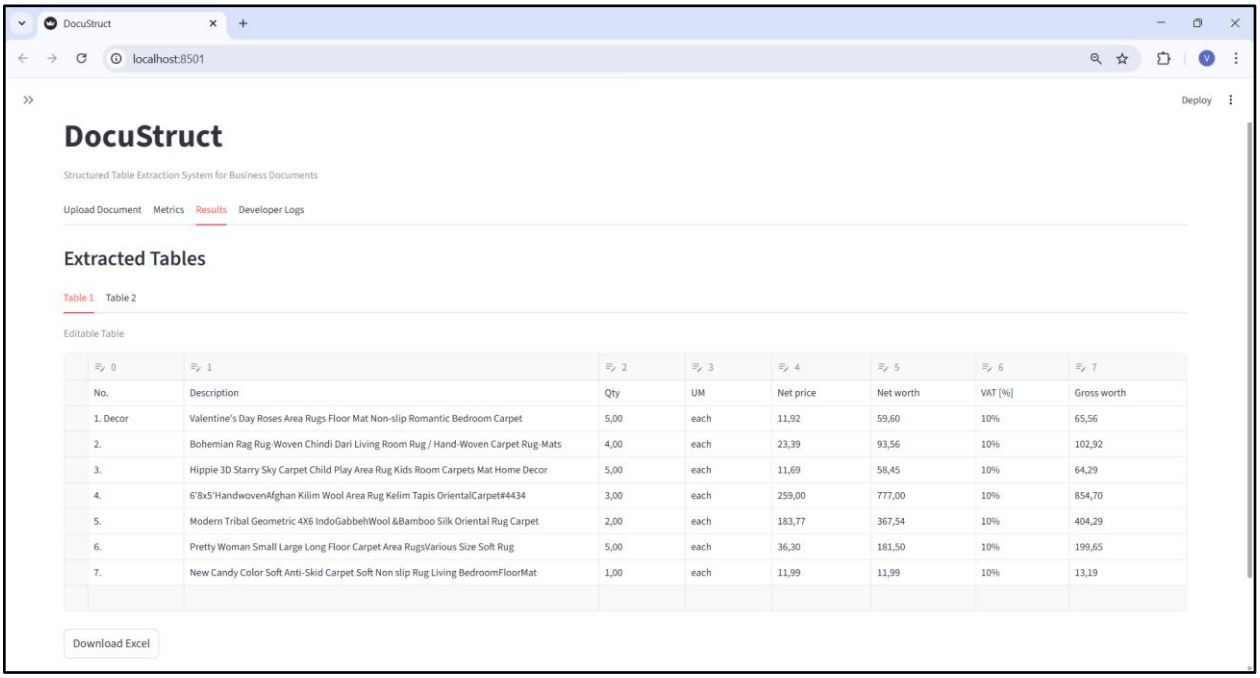


Screenshot 5: logs



Screenshot 6: Structured Table Output

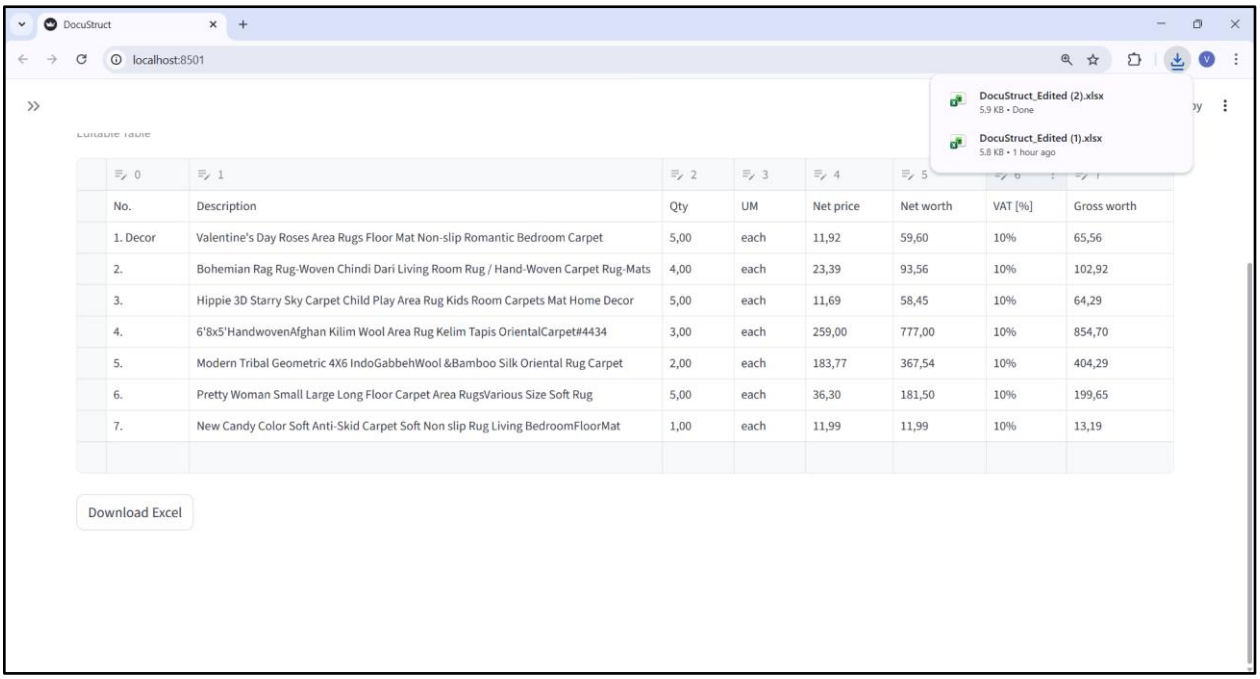
The reconstructed table is displayed in a structured format.



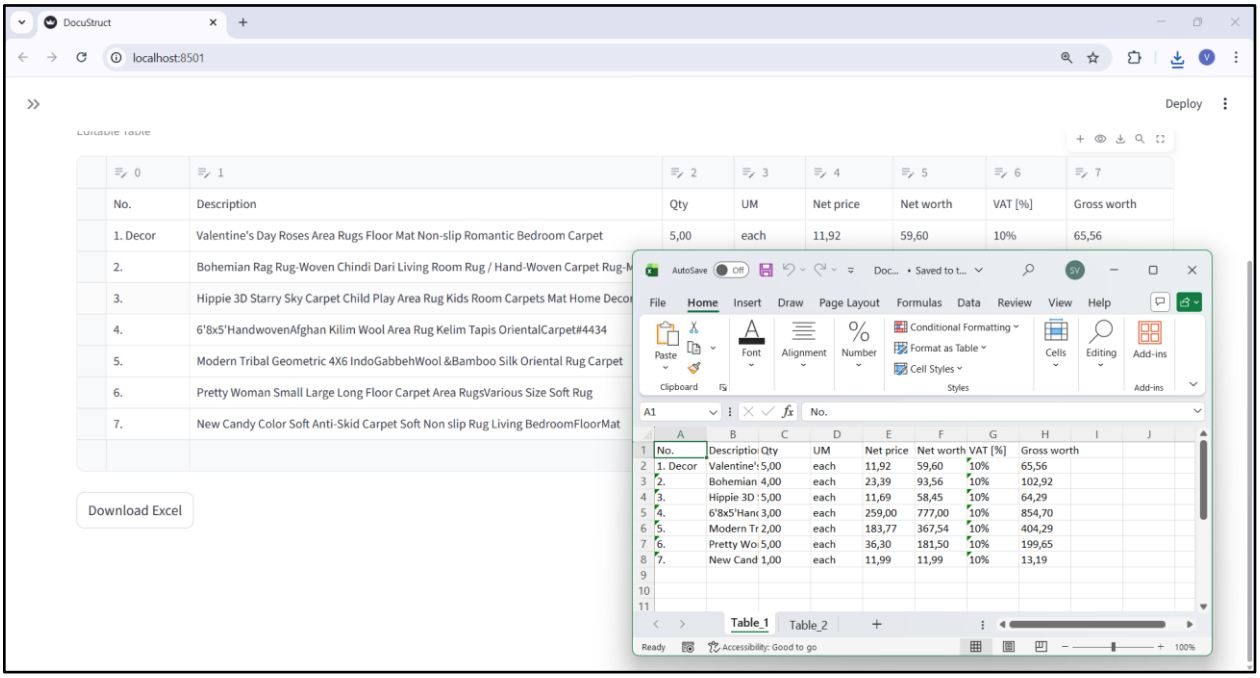
0	1	2	3	4	5	6	7
No.	Description	Qty	UM	Net price	Net worth	VAT [%]	Gross worth
1. Decor	Valentine's Day Roses Area Rugs Floor Mat Non-slip Romantic Bedroom Carpet	5,00	each	11,92	59,60	10%	65,56
2.	Bohemian Rag Rug-Woven Chindi Dari Living Room Rug / Hand-Woven Carpet Rug-Mats	4,00	each	23,39	93,56	10%	102,92
3.	Hippie 3D Starry Sky Carpet Child Play Area Rug Kids Room Carpets Mat Home Decor	5,00	each	11,69	58,45	10%	64,29
4.	6'8x5'HandwovenAfghan Kilim Wool Area Rug Kelim Tapis OrientalCarpet#4434	3,00	each	259,00	777,00	10%	854,70
5.	Modern Tribal Geometric 4X6 IndoGabbehWool &Bamboo Silk Oriental Rug Carpet	2,00	each	183,77	367,54	10%	404,29
6.	Pretty Woman Small Large Long Floor Carpet Area RugsVarious Size Soft Rug	5,00	each	36,30	181,50	10%	199,65
7.	New Candy Color Soft Anti-Skid Carpet Soft Non slip Rug Living BedroomFloorMat	1,00	each	11,99	11,99	10%	13,19

Screenshot 7: Excel Export Result

The extracted data is exported into an Excel file for further use.



No.	Description	Qty	UM	Net price	Net worth	VAT [%]	Gross worth
1. Decor	Valentine's Day Roses Area Rugs Floor Mat Non-slip Romantic Bedroom Carpet	5,00	each	11,92	59,60	10%	65,56
2.	Bohemian Rag Rug-Woven Chindi Dari Living Room Rug / Hand-Woven Carpet Rug-Mats	4,00	each	23,39	93,56	10%	102,92
3.	Hippie 3D Starry Sky Carpet Child Play Area Rug Kids Room Carpets Mat Home Decor	5,00	each	11,69	58,45	10%	64,29
4.	6'8x5'HandwovenAfghan Kilim Wool Area Rug Kelim Tapis OrientalCarpet#4434	3,00	each	259,00	777,00	10%	854,70
5.	Modern Tribal Geometric 4X6 IndoGabbehWool &Bamboo Silk Oriental Rug Carpet	2,00	each	183,77	367,54	10%	404,29
6.	Pretty Woman Small Large Long Floor Carpet Area RugsVarious Size Soft Rug	5,00	each	36,30	181,50	10%	199,65
7.	New Candy Color Soft Anti-Skid Carpet Soft Non slip Rug Living BedroomFloorMat	1,00	each	11,99	11,99	10%	13,19



Screenshot 8: Terminal

```
C:\Windows\System32\cmd.e

[12:39:18] Execution started
[12:39:18] Layout Detection

0: 640x480 8 Section-headers, 2 Tables, 3 Texts, 1174.9ms
Speed: 60.9ms preprocess, 1174.9ms inference, 31.3ms postprocess per image at shape (1, 3, 640, 480
)
[12:39:20] Tables detected: 2
[12:39:23] Processing table 1
[12:39:23] Table Extraction
--> PP-Structure
[12:39:35] Extraction engine used: pp_structure
[12:39:35] OCR Processing
[12:39:35] OCR words detected: 0
[12:39:35] Row Detection
[12:39:35] Row segments detected: 8
[12:39:35] Column Detection
[12:39:35] Columns detected: 8
[12:39:35] Table Construction
[12:39:35] Table size: 8 rows x 8 columns
[12:39:35] Confidence Analysis
[12:39:35] Applying PP-Structure advanced scoring
[12:39:35] OCR Metrics
[12:39:35] {'avg_confidence': 0, 'min_confidence': 0, 'low_confidence_words': 0, 'total_words': 0}
[12:39:35] Structure Metrics
[12:39:35] {'logical_row_count': 8, 'column_count': 8}
[12:39:35] Completeness Metrics
[12:39:35] {'total_cells': 64, 'empty_cells': 0, 'empty_cell_ratio': 0.0}
[12:39:35] Numeric Metrics
[12:39:35] {'numeric_cell_ratio': 0.6406, 'numeric_cells': 41}
[12:39:35] Overall Score: 0.973
[12:39:35] Status: High Reliability
[12:39:35] Processing table 2
[12:39:35] Table Extraction
-> Fallback to Structured logic
[12:39:37] Extraction engine used: custom
[12:39:37] OCR Processing
[12:39:37] OCR words detected: 12
[12:39:37] Row Detection
[12:39:37] Row segments detected: 3
[12:39:37] Column Detection
[12:39:37] Columns detected: 4
[12:39:37] Table Construction
[12:39:37] Table size: 1 rows x 4 columns
[12:39:37] Confidence Analysis
[12:39:37] OCR Metrics
[12:39:37] {'avg_confidence': 0.9933, 'min_confidence': 0.9723, 'low_confidence_words': 0, 'total_w
ords': 12}
[12:39:37] Structure Metrics
[12:39:37] {'logical_row_count': 3, 'column_count': 4}
[12:39:37] Completeness Metrics
[12:39:37] {'total_cells': 4, 'empty_cells': 1, 'empty_cell_ratio': 0.25}
[12:39:37] Numeric Metrics
[12:39:37] {'numeric_cell_ratio': 0.0, 'numeric_cells': 0}
[12:39:37] Overall Score: 0.9216
[12:39:37] Status: High Reliability
[12:39:37] Result Aggregation
[12:39:37] Total tables processed: 2
[12:39:37] Average score: 0.9473
[12:39:37] Final status: High Reliability
[12:39:37] Excel Export
Excel file saved at: C:\Users\SHIVAM~1\AppData\Local\Temp\output.xlsx
[12:39:37] Excel file generated successfully
[12:39:37] Execution completed
```

These screenshots demonstrate the functionality and effectiveness of the developed system in extracting structured information from invoice documents.

11.2 Conclusion

This research project focused on designing and implementing a system that automatically extracts structured tabular data from the invoice documents. The system combines document layout detection, optical character recognition, and spatial analysis techniques to rebuild table structures from invoice images.

The system shows that combining layout detection models with clustering-based structural analysis can effectively manage different invoice formats. Its modular pipeline architecture allows it to detect table regions, recognize text, reconstruct logical rows and columns, and export structured data into spreadsheet formats.

The system includes confidence analysis metrics, which give users useful insights into the reliability of the extraction results. This feature helps highlight cases where manual verification might be needed. The graphical user interface created with Streamlit makes the system user-friendly and suitable for demonstrations.

The current system has certain limitations when handling borderless or poorly structured invoices may lead to reduced extraction accuracy due to the absence of clear structural cues. Additionally, processing time can increase when multiple tables are present within a single document. At present, PDF support is limited to first-page analysis in order to maintain acceptable performance and response time.

There is significant scope for enhancement in future iterations. The system can be extended to support batch processing of multiple invoices in a single upload, enabling higher throughput. Furthermore, improvements in structural understanding and data reconstruction can help minimize information loss and logical inconsistencies, thereby increasing overall accuracy and reliability.

Overall, the project successfully shows that automated invoice table extraction is possible using current document analysis techniques.

12. REFERENCES

[1] B. Pfitzmann *et al.*, "DocLayNet: A large human-annotated dataset for document layout analysis," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2022.

[2] PaddlePaddle, "PaddleOCR Documentation," [Online]. Available: GitHub

[3] Y. Du, C. Li, R. Guo, X. Yin, W. Liu, Y. Zhou, and X. Bai, "PaddleOCR: An open-source OCR system," *arXiv preprint* arXiv:2009.09941, 2020.

[4] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016.

[5] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, 2015.

[6] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. Int. Conf. Document Anal. Recognit. (ICDAR)*, 2007.

[7] M. Ester, H. P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proc. 2nd Int. Conf. Knowl. Discovery Data Mining (KDD)*, 1996.

[8] G. Bradski, "The OpenCV Library," 2000.

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[10] T. Kieninger and A. Dengel, "The T-Recs table recognition and analysis system," in *Proc. Int. Workshop Document Anal. Syst.*, 1998.

[11] F. Shafait, D. Keysers, and T. M. Breuel, “Table detection in heterogeneous documents,” in *Proc. Int. Workshop Document Anal. Syst.*, 2010.